

T1.4.F — Cancelación al final de período

stripe.subscriptions.update(cancel_at_period_end=true) · acceso preservado hasta period_end

Owner: Celestino · 2026-05-04 · Branch **pr/t1.4.f-cancel-at-period-end** · Commit **a15652f**

Status: Branch pusheada desde main. tsc 0 errores · build OK · 16/16 static pages. Lint 14 (Δ -2 vs baseline).
Bloqueador de Dona Control resuelto: T1.4.E queda libre para mergear después de este PR. NO mergeado a main.

1. Archivos modificados

Archivo	Δ líneas	Tipo	Rol
landing/app/api/cancel-subscription/route.ts	+56 / -11	M	subscriptions.cancel() → subscriptions.update(cancel_
TOTAL	+56 / -11	1 archivo	

2. Cambio principal

Antes (T1.3 / pre-T1.4.F)

```
const canceled = await getStripe().subscriptions.cancel(subscriptionId);
return NextResponse.json({
  status: canceled.status,
  message: "Subscription canceled successfully",
});
```

Comportamiento: cancela la suscripción de inmediato. Stripe revoca el acceso al instante, los créditos del período actual se pierden si el flujo upstream no los respeta.

Después (T1.4.F)

```
const updated = await getStripe().subscriptions.update(
  subscriptionId,
  { cancel_at_period_end: true },
);

// current_period_end vive en items.data[0] en stripe@22.
const firstItem = updated.items?.data?.[0];
const periodEnd = typeof firstItem?.current_period_end === "number"
  ? firstItem.current_period_end : null;

return NextResponse.json({
  ok: true,
  cancel_at_period_end: Boolean(updated.cancel_at_period_end),
  current_period_end: periodEnd,
  status: updated.status,
  message:
    "Cancelación programada al final del período actual. "
    + "Mantienes acceso hasta esa fecha.",
});
```

Comportamiento: la suscripción queda marcada para no renovar. Stripe emite *customer.subscription.updated* (cap=true) inmediatamente y *customer.subscription.deleted* al llegar period_end. El usuario mantiene acceso y créditos hasta esa fecha.

3. Flujo end-to-end resultante

```
1. Usuario clic "Cancelar suscripción" en el dashboard
  ■
  ▼
2. POST /api/cancel-subscription (landing, server-side)
  ■ auth() obtiene subscriptionId de la sesión
  ■ (NO acepta IDs del cliente)
  ▼
3. stripe.subscriptions.update(id, {cancel_at_period_end: true})
  ■
  ▼
4. Stripe emite customer.subscription.updated (cap=true)
  ■
  ▼
5. Stripe webhook → landing /api/webhook → bridge HMAC
  → backend /internal/stripe-event
  → procesar_evento_suscripcion (T1.3.C)
  → SuscripcionStripe queda marked
  ■
  ■ [HASTA period_end: usuario sigue con acceso, créditos
  ■ intactos, dashboard muestra "Se cancela el DD/MM"
  ■ en amber con botón deshabilitado (T1.4.E)]
  ■
  ▼
6. Al period_end: Stripe emite customer.subscription.deleted
  → bridge → backend → _procesar_subscription_deleted
  → status='canceled' · saldo de créditos PRESERVADO
  (decisión owner T1.3.C: lo que pagó es suyo)
```

4. Respuesta del endpoint

Extendida para que el dashboard refresque y la UI muestre inmediatamente el estado nuevo (T1.4.E sabe leer estos campos):

```
{
  "ok": true,
  "cancel_at_period_end": true,    // → UI muestra texto amber
  "current_period_end": 1717545600, // timestamp Unix de Stripe items[0]
  "status": "active",              // sigue active hasta period_end
  "message": "Cancelación programada al final del período
              actual. Mantienes acceso hasta esa fecha."
}
```

Códigos HTTP

Status	Cuándo
401	Sin sesión NextAuth
400	Sesión sin subscriptionId
500	Stripe SDK falla — error genérico 'cancel_failed'
200	Cancelación programada exitosamente

5. Mejoras paralelas (sin cambiar la intención)

- **subscriptionId tipado:** *(session as { subscriptionId?: string })* en vez de *as any*. Confirmando guardrail T1.4.D #1: el ID viene exclusivamente de la sesión server-side.
- **shortId() para logs:** trunca el `subscription_id` en logs a `'sub_XXXXXXXX...abcd'` (guardrail #2 sobre IDs Stripe en producción).
- **Error genérico al cliente:** `'cancel_failed'` en vez de propagar `err.message` de Stripe — no filtra mensajes internos del SDK al frontend.
- **Códigos estables:** `'unauthenticated'`, `'no_subscription_in_session'`, `'cancel_failed'`. Coherentes con los códigos que devuelve `/api/dashboard-data` (T1.4.D).
- **Try/catch con unknown:** `catch (err: unknown)` en lugar de `any`; uso `err instanceof Error` para extraer mensaje.

6. Pruebas ejecutadas

Comando	Resultado
<code>npx tsc --noEmit</code>	0 errores
<code>npm run lint</code>	14 problems (11 errors, 3 warnings) · todos preexistentes en <code>auth.ts</code> / <code>app/page.tsx</code> / <code>app/layout.tsx</code> · 0 en el
<code>Δ</code> vs baseline 16	−2 errores: eliminé <code>`(session as any).subscriptionId`</code> y <code>`catch (err: any)`</code> que estaban en el archivo original
<code>npm run build</code>	✓ Compiled successfully · 16/16 static pages · <code>/api/cancel-subscription</code> sigue como <code>f</code> (dynamic, server-render

7. Confirmaciones (lo que NO se hizo)

- ✓ NO deploy.
- ✓ NO merge.
- ✓ NO env changes.
- ✓ NO se modificó backend (la lógica de preservar saldo en `subscription.deleted` ya estaba en T1.3.C).
- ✓ NO se escribe en DB.
- ✓ NO se modificó UI (T1.4.E ya soporta el flow visual).
- ✓ Sin secrets filtrados (logs truncan IDs; errores genéricos al cliente).
- ✓ Stripe Dashboard intacto.

8. Riesgos

Riesgo	Severidad	Notas
Cliente cancela y compra créditos paquete one-time antes de period_end	Alto	Sin impacto: flujos paralelos, los paquetes one-time no usan este endpoint
Webhook <code>subscription.updated</code> <code>cap=true</code> se demora	Bajo	Stripe override en T1.4.D consulta directo a Stripe SDK, muestra <code>cancel_at_period_end</code>
Stripe falla al update por estado inválido (sub ya cancelado)	Bajo	Cae al catch → <code>cancel_failed 500</code> al cliente; UX degrada limpio
Comportamiento distinto vs cancelaciones inmediatas por UI	Alto	Es el cambio deseado. Si hay usuarios en producción, el message del response
<code>current_period_end</code> null si Stripe no lo devuelve	Muy bajo	El dashboard maneja null y oculta la fecha; backend usa el webhook real para

9. Próximos pasos · orden de merge sugerido

- **1. Mergear T1.4.F** primero (*pr/t1.4.f-cancel-at-period-end @ a15652f*). Desbloquea T1.4.E según Dona Control.
- **2. Mergear T1.4.E** después (*pr/t1.4.e-dashboard-ui @ 15dc518*). UI ya lista para el nuevo flow de cancelación.
- Ambas branches partieron de main; sin conflictos esperables.

PR links:

- T1.4.F: <https://github.com/celestinoibm/Dona-agent/compare/main...pr/t1.4.f-cancel-at-period-end>
- T1.4.E: <https://github.com/celestinoibm/Dona-agent/compare/main...pr/t1.4.e-dashboard-ui>

Tras mergear ambos: tarde o temprano hará falta un housekeeping de los ~15 archivos untracked acumulados en docs/auditorias/ y docs/scripts/ desde T1.4.A. Cuando T1.4 cierre, branch *pr/post-t1.4-housekeeping*.